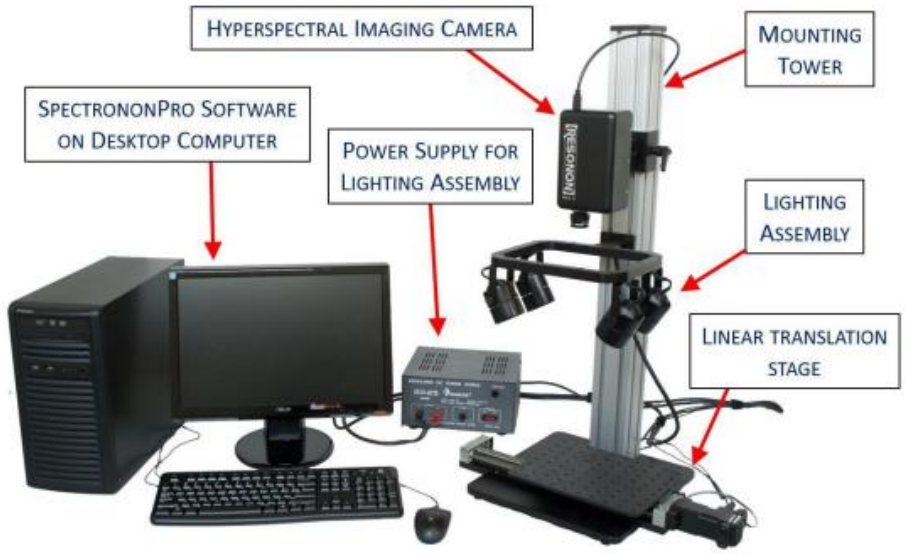
Mini-Project 2: CNN-Based Classification of Hyperspectral Polymer Images (ME3101-Autumn2026)

1. Background

A regular RGB image represents each pixel using three broad spectral channels: red, green and blue. A hyperspectral image records measurements across many narrow wavelength bands, providing a spectral signature at every pixel. These signatures may reveal material properties because different molecular structures absorb and reflect near-infrared radiation differently. This project investigates four polymers: high-density polyethylene (HDPE), polyethylene terephthalate (PET), polypropylene (PP) and polystyrene (PS).



The dataset was acquired using a Resonon Pika IR benchtop line-scan imaging system. Its 168-band near-infrared images combine spatial information (shape and texture) with spectral information potentially useful for material identification.

Imaging parameter	Specification
Wavelength range	900–1700 nm
Spectral bands / sampling	168 bands; approximately 4.8 nm
Detector / bit depth	TE-cooled InGaAs photodiode array; 14-bit
Spatial channels / pixel pitch	320 channels; 30 µm
Illumination	Four quartz-halogen lamps with diffused cylindrical sources; reported 32,767 lux
Acquisition	Line scan on 21 × 29 cm moving stage at 2 cm/s
Software	Spectral Scanner v2.3

2. Objective and expected workflow

Design, train and evaluate a convolutional neural network (CNN) that predicts the polymer class of a previously unseen hyperspectral image. Students are provided helper scripts for reading and

exploring the hyperspectral data; the principal assessed work is CNN design, training, controlled experimentation and interpretation of classification metrics.

Workflow: BIL + HDR → hyperspectral cube (H × W × 168) → 10 common PCA-selected original bands (H × W × 10) → padding only (HT × WT × 10) → MATLAB datastore → CNN → predicted polymer and evaluation metrics.

3. Dataset and supplied resources

The dataset has four class folders: HDPE, PET, PP and PS. Each contains .bil images and corresponding .hdr metadata. The folder determines the categorical class label. Images may have different numbers of rows and columns, but all contain 168 bands in a consistent spectral order. The .hdr files supply the metadata needed to decode the binary .bil files.

Provided helper	Purpose
readfile.m	Demonstrates reading a BIL/HDR pair, inspecting dimensions and displaying pseudo-RGB bands.
reduceHypercubePCA.m	Demonstrates PCA-based importance ranking and selection of 10 original spectral bands.

Important implementation rule: The supplied PCA function selects bands for one cube at a time. Students must not independently select different bands for different images. Determine one common set of 10 band indices using training data only, then apply those same indices to all training, validation, test and unseen images.

4. Tasks and deliverables

Task 1 — Read and inspect the hyperspectral data

- Use the supplied helper code and HDR metadata to load BIL images into H × W × 168 arrays.
- Derive class labels automatically from the parent folders.
- Inspect image sizes and visualize representative images and pixel spectral signatures.

Task 2 — Partition data and select 10 bands using PCA

- Create stratified training (70%), validation (15%) and test (15%) subsets, with a fixed random seed.
- If images share an original specimen or acquisition group, keep related images in the same subset to avoid leakage.
- Fit PCA using representative training spectra only; inspect explained variance and rank original bands using variance-weighted squared loadings.
- Select one common set of 10 original band indices and reuse it across all splits.

Task 3 — Standardize spatial size using padding only

- Determine common target dimensions HT and WT sufficient for the images; use image dimensions, not class labels, for this choice.
- Apply padding only, preferably balanced across opposing edges, to obtain HT × WT × 10 inputs.
- Do not crop, resize, interpolate, discard original pixels, or alter spectral-band ordering.
- Use a consistent padding value and explain its interaction with normalization; retain the original data unchanged.

- During inference, detect and report images exceeding the established target dimensions.

Task 4 — Construct MATLAB datastores

- Build training, validation and test datastores using a custom reader where needed for BIL/HDR data.
- Ensure that each observation yields a fixed-size 10-channel tensor and the correct categorical label.
- Verify datastore class counts and inspect example outputs.

Task 5 — Independently design and train a CNN (main design task)

- Design a 2D CNN accepting HT × WT × 10 inputs and producing four class scores.
- Include convolutional layers, nonlinear activations, batch normalization, suitable downsampling and a classification head.
- Choose depth, filter size/count, pooling, optimizer, learning rate, batch size, epochs and regularization.
- Use validation results to guide model selection; monitor training/validation loss and accuracy.
- Save the trained network, class ordering, selected band indices, padding dimensions and normalization parameters.

Task 6 — Investigate CNN design choices

- Perform at least two controlled comparisons, changing one factor at a time (e.g., filters/depth, learning rate, dropout or L2 regularization).
- Keep data partitions fixed and compare training/validation performance.
- Explain signs of underfitting, overfitting and optimization difficulties.

Task 7 — Evaluate the final CNN on held-out images (main analysis task)

- Generate a 4 × 4 confusion matrix with clearly labeled true and predicted classes.
- Calculate overall accuracy and class-wise precision, recall and F1-score for HDPE, PET, PP and PS.
- Calculate macro-averaged precision, recall and F1-score; also report weighted F1 if classes are imbalanced.
- Identify the most frequent class confusions and examine representative errors.
- Explain which metric matters most for an automated plastic-sorting application and why.

For each class, treat it as positive and all other classes as negative: Precision = TP/(TP+FP); Recall = TP/(TP+FN); F1 = 2×Precision×Recall/(Precision+Recall). State how undefined denominators are handled. Accuracy = number correct / total predictions.

Required evaluation questions

1. Which class has the highest precision and which has the highest recall?
2. Which classes are most frequently confused, and what spectral or visual factors might explain the confusion?
3. Can overall accuracy be high while recall for one polymer is poor? Explain using the confusion matrix.
4. When is macro F1 more informative than overall accuracy?
5. For an automated sorting line, would you prioritize precision, recall, F1 or accuracy? Justify using the costs of false positives (contaminating a recovered polymer stream) versus false negatives (rejecting recoverable material).

Task 8 — Predict the class of an unseen image

Create a MATLAB routine that accepts a new .bil/.hdr pair, applies the saved common band indices, normalization and fixed padding, then predicts HDPE, PET, PP or PS. Do not refit PCA or retrain the CNN during inference.

5. Final submission

1. **MATLAB source code (.m):** Submit all scripts and functions for data loading, PCA-based band selection, padding, datastore preparation, CNN training, evaluation and inference.
2. **README (.txt or .md):** State MATLAB/toolbox requirements, dataset source and folder layout, execution order, and configurable parameters.
3. **Trained model (.mat):** Save the final CNN as Model/trained_cnn.mat, ready to use without retraining.
4. **Preprocessing settings (.mat):** Save the common 10 selected band indices and PCA information, normalization parameters, target image size, padding method and class-label mapping.
5. **Technical report (.pdf):** Summarize the dataset, padding, CNN design, training, controlled experiments, confusion matrix, class-wise metrics and engineering interpretation of metric choice.
6. **Experimental results (.csv or .mat):** Provide results for the tested CNN configurations, including overall accuracy, average class-wise accuracy (mean recall), macro F1-score and other relevant metrics. Define each metric clearly.
7. **Result plots (.png or .jpg):** Save PCA explained variance, training curves, confusion matrix, class-wise metric plots and experiment comparisons under Results/Plots/, using descriptive filenames.
8. **Unseen-image demonstration (.m and .png):** Include Code/classify_unseen_image.m, identify the source of the unseen BIL/HDR image, and save a figure showing the predicted class and scores. A pixel-wise classification map is optional and only appropriate if pixel-level inference is implemented.
9. **ZIP organization:** Organize files under Code/, Model/, Results/Plots/ and Report/. Remove duplicates, temporary files and unnecessary dataset copies.
10. **Final ZIP:** Submit one file named Rollnumber_name_project_2.zip, maximum 25 MB. Ensure the submission can be opened and its results reproduced with the separately supplied dataset.